

Dự án cuối khoá — Đề 1: Quản lý quán cà phê

Em xây cái gì	API quản lý bán hàng cho một quán cà phê, chạy thật trên internet
Ai dùng	Chủ quán và nhân viên phục vụ, ngay trong ca làm, mở /docs trên điện thoại
Làm với ai	Một mình. Được dùng AI, nhưng lúc trả bài phải giải thích được mọi dòng
Bắt đầu khi nào	Sau buổi 12 — không đợi tới buổi 35
Nộp gì	4 thứ ở mục 1
Chấm	100 điểm: 60 máy chấm tự động, 40 giáo viên chấm tay, cộng tối đa +10

Quán thay quyển sổ ghi món bằng API của em: mở hoá đơn cho một bàn, gọi món, bỏ món, thanh toán, và cuối ngày chủ quán biết được doanh thu cùng món nào bán chạy.

Dự án gom lại gần như mọi thứ khoá đã dạy: vòng lặp, hàm, đọc ghi file, lớp và kế thừa, SQL, FastAPI, deploy. Mục 11 chia sẵn thành 9 mốc gắn với số buổi — học tới đâu làm tới đó.

1. Bốn thứ phải nộp

#	Nộp gì	Coi là xong khi
1	Link repo GitHub công khai	Người lạ clone về, làm theo README.md, chạy được
2	Link API chạy thật: https://...onrender.com/docs	Mở trên trình duyệt, bấm thử endpoint được ngay
3	thong-tin-nop.json ở gốc repo	Đúng mẫu ở mục 13, mọi endpoint khai trong đó gọi thử được
4	Kịch bản dùng thật, ~10 dòng trong README.md	Người chấm đọc xong hiểu ai dùng, dùng lúc nào, gọi những request nào

README.md phải có đủ bốn phần: **dự án làm gì cho ai** · **cách chạy trên máy** · **bảng endpoint** · **bản đồ kiến thức** (chỉ rõ chỗ nào là OOP, chỗ nào đọc ghi file, chỗ nào SQL, kèm số buổi).

2. Sáu mươi phút đầu tiên

Làm đúng sáu việc này trước khi gõ dòng code nào. Xong là em có bộ khung để bám suốt kỳ.

- [] **Viết kịch bản dùng thật** vào README.md — mục 3 có mẫu. Viết trước, code sau.
- [] **Vẽ 5 bảng ra giấy**, gạch mũi tên ngoại giữa chúng — mục 4.
- [] **Chọn 3 quy tắc nghiệp vụ** trong bảng ở mục 6, chép vào README.md.
- [] **Tạo repo GitHub** rỗng, commit README.md đầu tiên với ba thứ trên.
- [] **Tạo thư mục** theo đúng cây ở mục 12 — thư mục rỗng cũng được, để đúng chỗ ngay từ đầu.
- [] **Đọc mục 10** — cái bẫy của đề này. Biết trước thì thiết kế bảng đúng ngay từ lần đầu.

3. Kịch bản dùng thật

Một buổi làm việc có thật, viết thành dãy request. Đây là **ví dụ**; em viết kịch bản của riêng em. Kịch bản quyết định em cần đúng những endpoint nào — và tránh viết mười endpoint không ai gọi.

```
Sang sớm, nhân viên mở /docs, bấm Authorize, đăng nhập.
1. Khách gọi bàn 5          -> POST   /hoa-don          {"ban_id": 5}
2. Gọi 2 cà phê sữa đá     -> POST   /hoa-don/1/mon    {"mon_id": 2, "so_luong": 2}
3. Gọi thêm 1 bánh flan    -> POST   /hoa-don/1/mon    {"mon_id": 16, "so_luong": 1}
4. Khách đòi ý bỏ bánh    -> DELETE /hoa-don/1/mon/2
5. Thanh toán              -> POST   /hoa-don/1/thanh-toan -> tổng tiền, bàn 5 về trong
6. Cuối ngày xem doanh thu -> GET    /thong-ke/doanh-thu
7. Gửi báo cáo kế toán     -> GET    /nhap-xuat/doanh-thu.csv
```

Thước đo một endpoint tốt: nhân viên đang đứng sau quầy, một tay cầm điện thoại, khách đợi. Việc nào bắt họ bấm ba lần mới xong là thiết kế chưa đạt.

4. Bảng dữ liệu

Năm bảng gợi ý. Thêm cột hay đổi tên tùy ý, miễn đủ chạy kịch bản của em.

Bảng	Cột chính	Ghi chú
mon	mã món, tên, nhóm, giá, <code>con_ban</code>	mã món <code>unique</code>
ban	số bàn, sức chứa, đang có khách hay không	
hoa_don	bàn nào, giờ mở, giờ đóng, tổng tiền, đã thanh toán chưa	1-n với <code>chi_tiet_hoa_don</code>
chi_tiet_hoa_don	hoá đơn nào, món nào, số lượng, <code>don_gia</code>	cột <code>don_gia</code> là cái bẫy — đọc mục 10
nguoi_dung	tên đăng nhập, mật khẩu đã bấm, vai trò	

5. Chức năng và tiêu chí nghiệm thu

Đọc bảng này theo lối: "làm xong chức năng đó nghĩa là gọi request kia thì phải ra đúng cái đang ghi trong cột bên phải". Mỗi dòng viết được thành một `assert` trong `tests/`.

Là ai, muốn gì	Gọi gì	Xong khi
Khách chưa đăng nhập, xem quán bán gì	GET /mon	200 + danh sách, không cần token
Nhân viên, mở hoá đơn cho bàn	POST /hoa-don	201 + hoá đơn mới, bàn 5 chuyển sang "đang có khách"
Nhân viên, gọi món vào hoá đơn	POST /hoa-don/{id}/mon	201, dòng chi tiết có lưu don_gia lúc gọi
Nhân viên, khách đổi ý bỏ món	DELETE /hoa-don/{id}/mon/{ct_id}	204, tổng tiền tính lại đúng

Là ai, muốn gì	Gọi gì	Xong khi
Nhân viên, thanh toán	POST /hoa-don/{id}/thanh-toan	200 + tổng tiền tính lại từ chi tiết, hoá đơn đóng, bàn về trống
Quản lý, sửa giá một món	PUT /mon/{id}	200 . Nhân viên gọi cùng request → 403
Quản lý, xem doanh thu	GET /thong-ke/doanh-thu	200 . Nhân viên gọi → 403
Quản lý, nhập menu mùa mới	POST /nhap-xuat/thuc-don.csv	200 + số dòng nhận / số dòng bỏ kèm lý do
Kế toán, lấy file doanh thu	GET /nhap-xuat/doanh-thu.csv	200 , tải về được (Content-Disposition)
Ai cũng gọi được, kiểm tra API sống	GET /health	200 , có chạm database

Ba mức quyền, phân biệt cho đúng:

	Xem thực đơn	Bán hàng	Sửa thực đơn, đổi giá	Xem doanh thu
Chưa đăng nhập	✓	401	401	401
Nhân viên	✓	✓	403	403
Quản lý	✓	✓	✓	✓

6. Quy tắc nghiệp vụ — phần đáng giá nhất của đề

Chọn **ít nhất ba** dòng dưới đây (hoặc tự nghĩ thêm). Mỗi quy tắc trả 409 kèm câu thông báo đọc là hiểu. Đây là thứ phân biệt một dự án thật với một bài tập nổi bằng.

#	Chặn khi	Vì sao chặn	Tự kiểm
1	Bàn đang có hoá đơn chưa thanh toán mà mở hoá đơn thứ hai	Lúc thanh toán không biết tính cái nào	Mở 2 hoá đơn cho bàn 5 → lần hai 409
2	Hoá đơn đã thanh toán mà còn thêm món	Khách trả tiền đi rồi, ghi thêm là sai số	Thanh toán xong gọi thêm món → 409
3	Món đang tắt con_ban mà vẫn gọi	Hết nguyên liệu, không để nhân viên gọi nhầm	Tắt món 2, gọi món 2 → 409
4	Hoá đơn chưa có món nào mà thanh toán	Tạo một dòng doanh thu bằng 0, làm bẩn báo cáo	Mở hoá đơn rồi thanh toán ngay → 409

Sau khi một request trả 409, request tiếp theo **vẫn phải chạy bình thường**. Ra 500 là em quên `db.rollback()`.

7. OOP viết vào đâu

Câu hỏi hay gặp nhất: "*biết viết class rồi, nhưng đặt nó vào đâu trong dự án web?*"

Nhìn lại bốn quy tắc ở mục 6. Viết thẳng thì em có một hàm `thanh_toan` dài với bốn `if` nối nhau, và thêm quy tắc mới là phải mở đúng hàm đó ra sửa. Thay vào đó, mỗi quy tắc là một **lớp con**; phần bán hàng chỉ duyệt danh sách và gọi lần lượt.

Chép khung này vào `app/nghep_vu.py` rồi điền:

```
from abc import ABC, abstractmethod

class QuyTac(ABC):
    """Mot quy tac chan mot thao tac. None = cho phép, chuoì = lý do chan."""

    @abstractmethod
    def kiem_tra(self, boi_canh: dict) -> str | None: ...

class BanDangCoKhach(QuyTac):
    def kiem_tra(self, boi_canh):
        ...

class HoaDonDaThanhToan(QuyTac):
    def kiem_tra(self, boi_canh):
        ...

DANH_SACH_QUY_TAC: list[QuyTac] = [BanDangCoKhach(), HoaDonDaThanhToan()]

def kiem_tra_tat_ca(boi_canh: dict) -> str | None:
    """Duyet moi quy tac, gap loi dau tien thi tra ve ngay."""
    ...
```

Chỗ thứ hai còn rõ hơn — **cách tính tiền**. Quán chạy khuyến mãi theo tháng: tháng này giảm 10%, tháng sau mua hai tặng một, tháng sau nửa về giá thường.

```
class CachTinhGia(ABC):
    @abstractmethod
    def tinh(self, don_gia: int, so_luong: int) -> int: ...

class GiaThuong(CachTinhGia): ...
class GiamTheoPhanTram(CachTinhGia): ...
class MuaHaiTangMot(CachTinhGia): ...
```

Đổi khuyến mãi = đổi một dòng cấu hình, phần tính tiền không sửa một chữ. Đó là đa hình, và đó là thứ người chấm tìm.

Tối thiểu: 1 lớp trừu tượng (`ABC` + `@abstractmethod`) + ≥ 2 lớp con, trong file riêng, không phải model SQLAlchemy. **Thử:** bỏ lớp đó đi mà code không xấu hơn thì nó chưa cần tồn tại.

8. Đọc ghi file

Chủ quán soạn menu mùa mới trên Excel rồi lưu CSV. Em làm ba endpoint:

Endpoint	Việc	Bắt buộc
POST /nhap-xuat/thuc-don.csv	Nhận UploadFile, ghi hàng loạt vào database	Trả về số dòng nhận / số dòng bỏ kèm lý do từng dòng
GET /nhap-xuat/doanh-thu.csv	Xuất doanh thu theo khoảng ngày	Có Content-Disposition: attachment
GET /nhap-xuat/bao-caocao.json	Xuất báo cáo tổng hợp	Có Content-Disposition: attachment

File thật không bao giờ sạch. Một dòng hỏng không được làm hỏng cả lần nhập — bỏ dòng đó, ghi lý do, nhập tiếp, và **tuyệt đối không trả 500**. Những thứ phải chịu được:

- [] dòng trống ở cuối file
- [] ô giá ghi 25.000 thay vì 25000
- [] mã món trùng nhau trong cùng file
- [] thiếu hẳn một cột
- [] file không phải UTF-8

Để phần này trong `app/nhap_xuat.py`, và module đó **chỉ làm việc với chuỗi và list/dict — không import FastAPI, không import SQLAlchemy**:

```
def doc_thuc_don(noi_dung_csv: str) -> tuple[list[dict], list[str]]:
    """Tra ve (danh sach mon doc duoc, danh sach ly do bo dong)."""
    ...
```

Ký kiểu như vậy thì test được bằng một chuỗi CSV gõ tay, không cần server, không cần database.

9. SQL báo cáo

Tính bằng **SQL**, không kéo hết dữ liệu về Python rồi cộng bằng vòng lặp.

Báo cáo	Phải dùng	Cấm
Doanh thu từng ngày trong một khoảng	<code>text()</code> viết tay, tham số <code>:ten_tham_so</code>	Nối chuỗi vào câu SQL
Top món bán chạy	<code>JOIN</code> chi tiết với món + <code>GROUP BY</code> + <code>SUM</code>	Kéo hết bảng về rồi đếm bằng Python

Câu viết tay bằng `text()` phải truyền tham số bằng `:ten_tham_so`. Nối chuỗi vào câu SQL là mở cửa cho SQL injection — lỗi bảo mật kinh điển.

10. Cái bẫy của đề này

Đơn giá phải được chép vào dòng hoá đơn ngay lúc khách gọi món.

Nhiều bạn làm sai: bảng `chi_tiet_hoa_don` chỉ lưu `mon_id` và `so_luong`, giá thì lúc cần lại tra sang bảng `mon`. Cách đó chạy đúng cho tới ngày chủ quán tăng giá cà phê — ngay hôm đó, hoá đơn của ba tháng trước cũng tự đổi theo giá mới, sổ sách sai hết.

Test bắt buộc phải có, ba bước:

1. Gọi 1 món giá 25000
2. Sửa giá món đó lên 50000
3. Thanh toán -> tổng tiền phải là 25000, không phải 50000

Bấy thứ hai nhẹ hơn: thanh toán là **ba việc trong một transaction** — tính lại tổng tiền, đóng hoá đơn, trả bàn về trống. Hồng giữa chừng mà đã commit một nửa thì bàn kẹt mãi ở trạng thái "đang có khách", không ai mở hoá đơn cho bàn đó được nữa.

11. Lộ trình 9 mốc

Bốn mốc đầu làm bằng **Python thuần, chưa cần biết FastAPI** — và đó lại là những phần khó sửa nhất nếu để dồn tới cuối.

Mốc	Sau buổi	Làm gì	Xong khi
1	12	Kịch bản dùng thật, vẽ 5 bảng, hàm <code>tinh_tien_dong(don_gia, so_luong)</code>	<code>python</code> chạy hàm đó ra số đúng với 3 trường hợp thử
2	15	<code>nhap_xuat.py</code> : đọc chuỗi CSV bản → <code>list[dict]</code> , ghi CSV, ghi JSON	Gõ tay một chuỗi CSV có 2 dòng rác, hàm trả đúng số dòng tốt và 2 lý do
3	19	(không bắt buộc) Mở CSV vừa xuất bằng Pandas để đối chiếu	Tổng doanh thu Pandas tính ra khớp số API trả
4	24	<code>nghepvu.py</code> : <code>QuyTac</code> + các lớp con, <code>CachTinhGia</code> + 3 lớp con	Chạy bằng dữ liệu giả (dict), mỗi quy tắc chặn đúng trường hợp của nó
5	29	API tạm giữ dữ liệu trong list: <code>GET/POST /mon</code> , schema Pydantic	<code>/docs</code> mở được, <code>POST</code> sai kiểu ra 422
6	30	Viết tay 2 câu SQL báo cáo, chạy trên SQLite	Cả hai câu trả đúng số khi đối chiếu bằng tay
7	33	<code>models.py</code> + Alembic + <code>crud.py</code> + <code>APIRouter</code> . Nối <code>nghepvu.py</code> vào luồng bán hàng	<code>alembic upgrade head</code> từ repo trống chạy được; kịch bản mục 3 chạy hết
8	34	Bảng <code>nguoi_dung</code> , <code>bcrypt</code> , JWT có <code>exp</code> , hai vai trò	Bảng phân quyền ở mục 5 đúng cả 12 ô
9	35	Deploy Render + PostgreSQL, <code>/health</code> , <code>README.md</code> , <code>thong-tin-nop.json</code>	Người khác mở link, đăng nhập, chạy hết kịch bản

Nộp giữa kỳ: sau buổi 12 (mốc 1) · sau buổi 15 (mốc 2) · sau buổi 24 (mốc 4) · sau buổi 33 (mốc 7) · **buổi 35 + 3 ngày: bản deploy đầu tiên** dù chưa xong tính năng · buổi 35 + 2 tuần: nộp đủ.

Mốc deploy cố tình đặt sớm. Deploy luôn phát sinh vấn đề không đoán trước; để tới hôm cuối mới đụng vào là không kịp.

12. Hợp đồng với bộ chấm — bảy điều phải đúng từng chữ

Có một bộ chấm tự động chạy trên bài của em. Nó **không đọc code để đoán ý, nó gọi API**. Ngoài bảy điều này, em tự do thiết kế.

1. Cấu trúc thư mục

```

app/main.py          <- co bien ten `app` (FastAPI)
app/models.py        <- model SQLAlchemy
app/schemas.py      <- schema Pydantic
app/routers/         <- it nhat 2 file router
tests/               <- test cua em, chay bang `pytest -q`
alembic/versions/    <- it nhat 2 migration
requirements.txt     <- ghim phien ban bang ==
.env.example         <- co, va KHONG co file .env trong repo
README.md
thong-tin-nop.json

```

2. Cấu hình đọc từ biến môi trường bằng `pydantic-settings`, đúng bốn tên: `CHUOI_KET_NOI`, `KHOA_BI_MAT`, `SO_PHUT_HET_HAN`, `TAO_DU_LIEU_MAU`. Bộ chấm chạy bài trên SQLite với `TAO_DU_LIEU_MAU=true`; dữ liệu mẫu phải tự nạp lúc khởi động, không cần chạy tay lệnh nào.

3. Tạo bảng bằng Alembic. `alembic upgrade head` chạy được từ thư mục gốc, và trong `app/` không còn `Base.metadata.create_all()` ở đâu nữa.

4. Đăng nhập theo chuẩn OAuth2 password form

```

POST /auth/dang-nhap    form: username, password  -> {"access_token": "...", "token_type": "bearer"}
GET  /auth/toi         Bearer token              -> thong tin nguoi dang nhap

```

Sai mật khẩu trả `401`. Không response nào được chứa mật khẩu hay mã băm.

5. `GET /health` trả `200` và có chạm database.

6. Endpoint danh sách — cái khai trong `thong-tin-nop.json` — có ba tham số: `?tim=` (không thấy thì trả `[]`), `?bo_qua=`, `?gioi_han=`. Riêng `gioi_han` phải ràng buộc `ge/le`, truyền `99999` vào ra `422`.

7. Hai vai trò. Ghi mà không kèm token → `401`. Tài khoản quyền thấp → `403`. Hai mã này khác nhau, đừng lẫn.

13. `thong-tin-nop.json`

```

{
  "ho_ten": "Nguyen Van A",
  "lop": "Python46",
  "de_tai": "de-01-quan-ca-phe",
  "link_repo": "https://github.com/tai-khoan-cua-em/du-an-cuoi-khoa",
  "link_api": "https://quan-ca-phe.onrender.com/docs",
  "tai_khoan": {
    "quyen_cao": {"ten_dang_nhap": "quanly", "mat_khau": "quanly123"},
    "quyen_thap": {"ten_dang_nhap": "nhanvien", "mat_khau": "nhanvien123"}
  },
  "endpoint": {
    "danh_sach": "/mon",
    "tao_moi": {
      "duong_dan": "/mon",
      "than": {"ma_mon": "ZZ99", "ten": "Mon thu", "nhom": "Khac", "gia": 15000}
    }
  },

```

```

    "xuat_csv": "/nhap-xuat/thuc-don.csv",
    "nhap_csv": "/nhap-xuat/thuc-don.csv",
    "thong_ke": "/thong-ke"
  },
  "quy_tac_409": [
    "Ban dang co hoa don mo thi khong mo hoa don thu hai",
    "Hoa don da thanh toan thi khong them mon",
    "Mon dang tat con_ban thi khong ban"
  ]
}

```

Hai tài khoản đó **phải có sẵn trong dữ liệu mẫu**: quyền cao để bộ chấm thử thao tác ghi, quyền thấp để thử xem có bị chặn 403 không. `tao_moi.than` là một body hợp lệ để gửi thử. Endpoint cần query param thì viết đầy đủ: `{"duong_dan": "/nhap-xuat/abc.csv", "tham_so": {"nam": 2026}}`.

14. Bảng kiểm kỹ thuật

Tick hết bảng này là chắc chắn không mất điểm oan ở phần A.

OOP

- [] ≥ 2 lớp của riêng em, **không phải** model SQLAlchemy, đặt trong file riêng
- [] Có `ABC` + `@abstractmethod` và ≥ 2 lớp con
- [] Phần còn lại của code gọi qua lớp cha, không cần biết đang chạy lớp con nào

Đọc ghi file

- [] 1 endpoint xuất CSV tải về được · 1 endpoint nhận `UploadFile` · 1 endpoint xuất JSON
- [] Module xử lý file không import FastAPI, không import SQLAlchemy
- [] Chịu được mọi loại file bắn ở mục 8

Dữ liệu và SQL

- [] ≥ 3 bảng nghiệp vụ + bảng `nguoi_dung`
- [] ≥ 1 quan hệ 1-n khai bằng `ForeignKey` + `relationship` + `back_populates`
- [] `unique` cho mã nghiệp vụ · `nullable=False` cho trường bắt buộc · `index` cho cột hay tìm và cột khoá ngoại
- [] ≥ 1 truy vấn `JOIN` + `GROUP BY` + hàm tổng hợp
- [] ≥ 1 câu SQL viết tay qua `text()` có tham số hoá
- [] ≥ 2 migration Alembic · `seed.py` bật tắt bằng `TAO_DU_LIEU_MAU`

API

- [] CRUD đầy đủ cho ≥ 2 bảng
- [] Dùng đúng 200 201 204 401 403 404 409 422
- [] Tìm kiếm và phân trang theo đúng mục 12
- [] ≥ 1 endpoint trả dữ liệu quan hệ lồng nhau, có `selectinload`
- [] Endpoint chia theo `APIRouter`, mỗi nhóm một file

Nghệ nghiệp

- [] ≥ 3 quy tắc, mỗi quy tắc trả 409 với thông báo đọc là hiểu
- [] Thao tác dụng nhiều bảng nằm trong một transaction

- [] Sau một 409 , request kế tiếp vẫn chạy bình thường

Xác thực

- [] Mật khẩu băm bằng bcrypt , không bao giờ ra ngoài
- [] JWT có exp · hai vai trò · phân biệt 401 với 403
- [] Đọc thì mở, ghi thì phải có quyền

Chất lượng và deploy

- [] app/config.py dùng pydantic-settings , mọi bí mật đọc từ biến môi trường
- [] .env không vào Git, .env.example thì có · requirements.txt ghim ==
- [] ≥8 test pytest dùng dependency_overrides + SQLite trong RAM, trong đó ≥3 test cho 409 và 1 test cho đọc file
- [] Chạy thật trên Render hoặc tương đương với PostgreSQL, có /health , alembic upgrade head nằm trong lệnh khởi động
- [] Deploy lần thứ hai không mất dữ liệu · không dùng Docker

Dữ liệu mẫu như thật

- [] Tên món có thật, giá đúng thị trường, 15–20 món, vài chục hoá đơn rải trong vài tuần
- [] Không có abc , test , mon1 — người chấm nhìn phát đầu tiên là biết dự án đã từng được mở ra dùng hay chưa

15. Chấm điểm

Phần A — bộ chấm tự động, 60 điểm. Chạy thẳng trên repo, không cần internet.

Nhóm	Điểm	Kiểm cái gì
Cấu trúc và chạy được	8	đủ file bắt buộc, ≥2 router · alembic upgrade head chạy · /health trả 200
Alembic và dữ liệu	6	≥2 migration · không còn create_all
OOP	8	lớp trừu tượng ở module riêng · ≥2 lớp con
Đọc ghi file	10	module thuần · xuất CSV · tải về được · file bản không làm chết API
SQL	6	SQL viết tay tham số hoá · GROUP BY + hàm tổng hợp · selectinload
API	8	phân trang · ràng buộc ge / le cho 422 · tìm kiếm · thống kê trả 200
Bảo mật	8	không token thì 401 · quyền thấp thì 403 · không lộ mật khẩu · có bcrypt, không commit .env
Kiểm thử	6	pytest ≥ 8 test pass · ≥3 test chạm 409 · có test cho phần đọc file

Phần B — giáo viên chấm tay, 40 điểm.

Nhóm	Điểm	Chấm cái gì
Chạy thật	10	link API mở được, /docs đủ endpoint, deploy lại lần hai dữ liệu còn nguyên, dùng PostgreSQL
Ý nghĩa thực tế	10	kịch bản dùng thật chạy trôi, dữ liệu mẫu như thật, endpoint bám đúng việc

Nhóm	Điểm	Chấm cái gì
OOP và nghiệp vụ	12	lớp có lý do tồn tại, thêm lớp con mới không phải sửa hàm cũ, ba quy tắc chặn đúng, thao tác nhiều bảng trong một transaction
Trình bày	8	README.md đủ bốn phần, có bản đồ kiến trúc, tên hàm tên biến nhất quán, không file rác

Cộng, tối đa +10 — trang HTML tĩnh gọi API (+4, và đây là lúc CORS thật sự có ích) · người ngoài lớp dùng thật kèm ảnh chụp hoặc lời xác nhận (+3) · GitHub Actions chạy `pytest` mỗi lần push (+2) · phân trang trả kèm tổng số bản ghi (+1).

Trừ — bí mật thật nằm trong repo, ví dụ khoá JWT hay mật khẩu database (−20) · SQLite trên production hoặc deploy lại là mất dữ liệu (−15) · còn `Base.metadata.create_all()` (−10) · mật khẩu lưu chữ thường hoặc lộ ra trong một response (−10) · lớp OOP dựng cho có, một lớp con hoặc không ai gọi qua lớp cha (−8) · dữ liệu mẫu kiểu `test`, `abc` (−5) · code không chạy được theo đúng README.md (−5).

Xếp loại. ≥90: đem đi phỏng vấn được, đưa người thật dùng được. ≥75: đạt yêu cầu một dự án thật. ≥60: chạy được nhưng thiếu chiều sâu. <60: làm lại phần thiếu, nộp lần hai.

16. Tự kiểm tra khi nộp

Clone repo của chính mình về **một thư mục trống** rồi làm đúng theo README.md :

```
git clone <link repo của em> thu-muc-trong && cd thu-muc-trong
pip install -r requirements.txt
cp .env.example .env
alembic upgrade head
fastapi dev app/main.py          # mở /docs, chạy hết kịch bản mục 3
pytest -q                        # >= 8 test, tất cả xanh
```

Rồi soát từng dòng:

- [] `grep -rn "create_all" app/` — không ra kết quả nào
- [] `alembic history` — có ít nhất hai bước
- [] `git ls-files | grep "\.env$"` — không ra gì
- [] Repo không còn `*.db`, `__pycache__`, `.venv`
- [] Thêm một lớp con thứ ba vào `nhiep_vu.py` — **không phải sửa hàm cũ** nào
- [] Xuất một CSV rồi mở bằng Excel — tiếng Việt không lỗi font
- [] Nhập một CSV **cố tình làm bẩn** (thiếu cột, dòng trống, dòng trùng, chữ ở ô số) — không ra 500
- [] Thử từng quy tắc mục 6 — ra đúng 409, và request ngay sau đó vẫn 200
- [] Chạy test ba bước ở mục 10 — tổng tiền tính theo giá **lúc gọi**
- [] Gọi mọi endpoint ghi không kèm token → 401; bằng tài khoản quyền thấp → 403
- [] Deploy lại lần hai rồi gọi API — dữ liệu còn nguyên
- [] Mọi endpoint khai trong `thong-tin-nop.json` gọi thử được trên link thật

17. Hỏi đáp nhanh

Làm nhóm được không? Không. Để đã cân cho một người.

Em muốn tự chọn đề tài khác. Được, và thường ra dự án tốt hơn vì em hiểu bài toán. Gửi trước ba thứ để duyệt: ai dùng và dùng lúc nào · sơ đồ bảng (vẽ tay chụp ảnh cũng được) · ba quy tắc nghiệp vụ định chặn. Đề của em phải đáp ứng đủ mục 14; để quá đơn giản sẽ bị trả lại kèm gợi ý mở rộng.

Chưa học tới FastAPI thì làm được phần nào? Mốc 1-4 ở mục 11, bằng Python thuần.

Không nghĩ ra quy tắc nghiệp vụ. Tự hỏi: thao tác nào người dùng làm sai được, và làm sai thì dữ liệu hỏng thế nào? Mỗi câu trả lời là một quy tắc.

Không biết đặt lớp OOP vào đâu. Tìm trong code một chỗ có nhiều `if` nối nhau cùng trả lời một câu hỏi: được phép không, tính bao nhiêu tiền, hạng nào. Mỗi nhánh `if` là một lớp con.

Render bắt trả tiền? Gói miễn phí đủ dùng. Railway, Fly.io, PythonAnywhere đều được, miễn là có link chạy thật và database không phải SQLite.

API ngủ đông, người chấm mở lên chờ lâu. Bình thường với gói miễn phí. Ghi một dòng trong `README.md` : lần gọi đầu có thể mất 30-60 giây để đánh thức.

18. Làm thêm nếu còn thời gian

Trang HTML tĩnh hiện thực đơn và bàn nào đang có khách (+4 điểm) · gộp hoá đơn khi khách chuyển bàn · thống kê giờ cao điểm trong ngày để chủ quán xếp ca.